Coreset Technique

(Part 2)

OUTLINE

1. Coresets and Composable Coresets (Part 1)
2. Case study: k-center clustering (Part 1)
3. Other applications
    - k-center as a coreset primitive
        - Diameter
        - Diversity maximization
    - k-means/median clustering

k-center as a coreset primitive

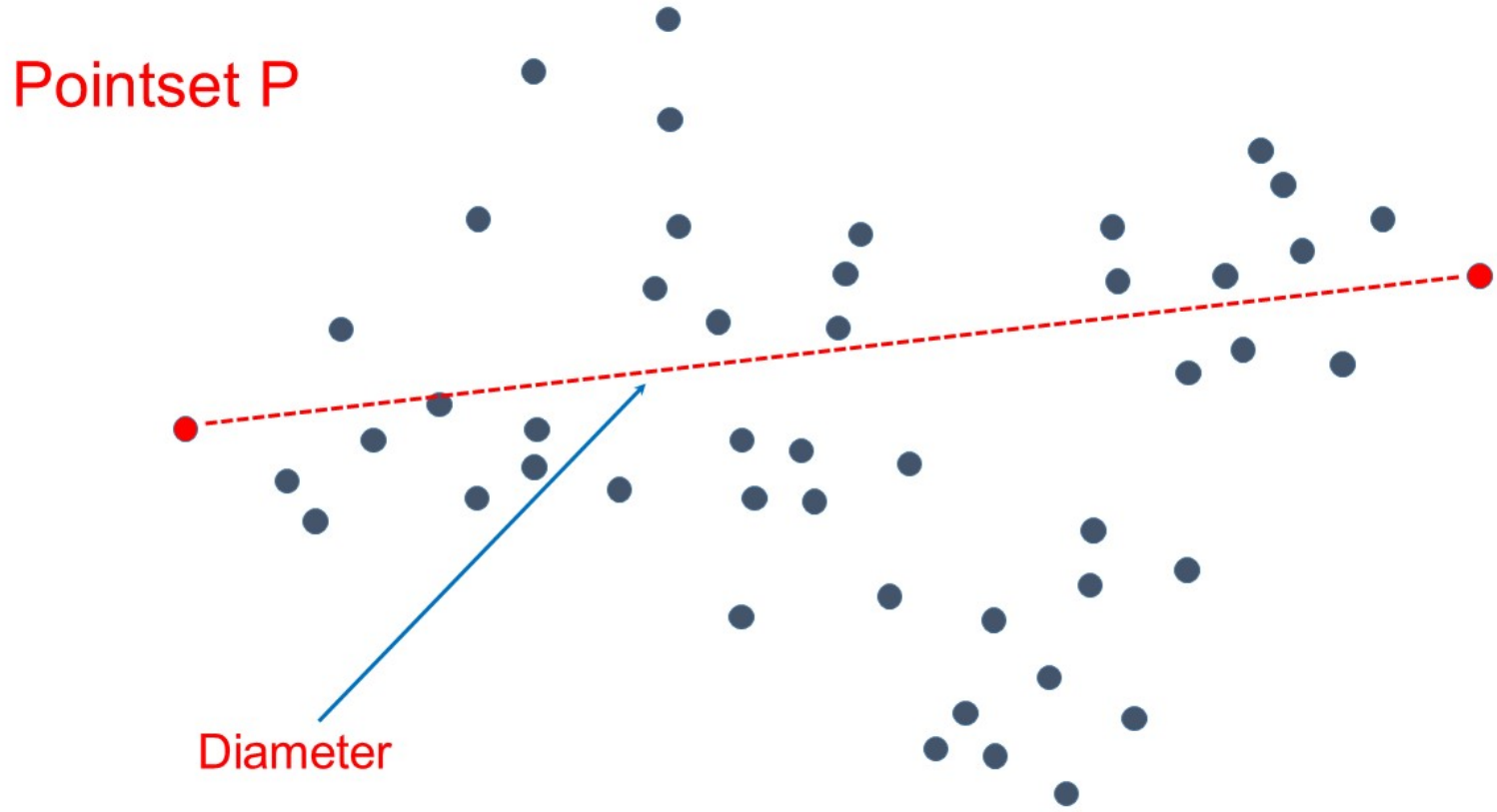
Diameter

The diameter of a set P of N points from a metric space (M, d) is the maximum distance between two points, i.e.,

$$\Delta(P) = \max_{x,y \in P} d(x, y),$$

Many applications: e.g.,

- Shape analysis: used in computational geometry, visualization, robotics
- Network analysis: used in social network analysis, biology, etc.


Pointset P
Diameter

Exact diameter computation

Sadly, the computation of the exact diameter requires almost quadratic operations (except for special cases such as the low-dimensional Euclidean spaces), hence it is impractical for large, high-dimensional pointsets.

Exercise

Design an MR algorithm to compute the exact diameter of a set P of N points, which requires $R = O(1)$, $M_L = O\left(\sqrt{N}\right)$ and $M_A = O\left(N^2\right)$.

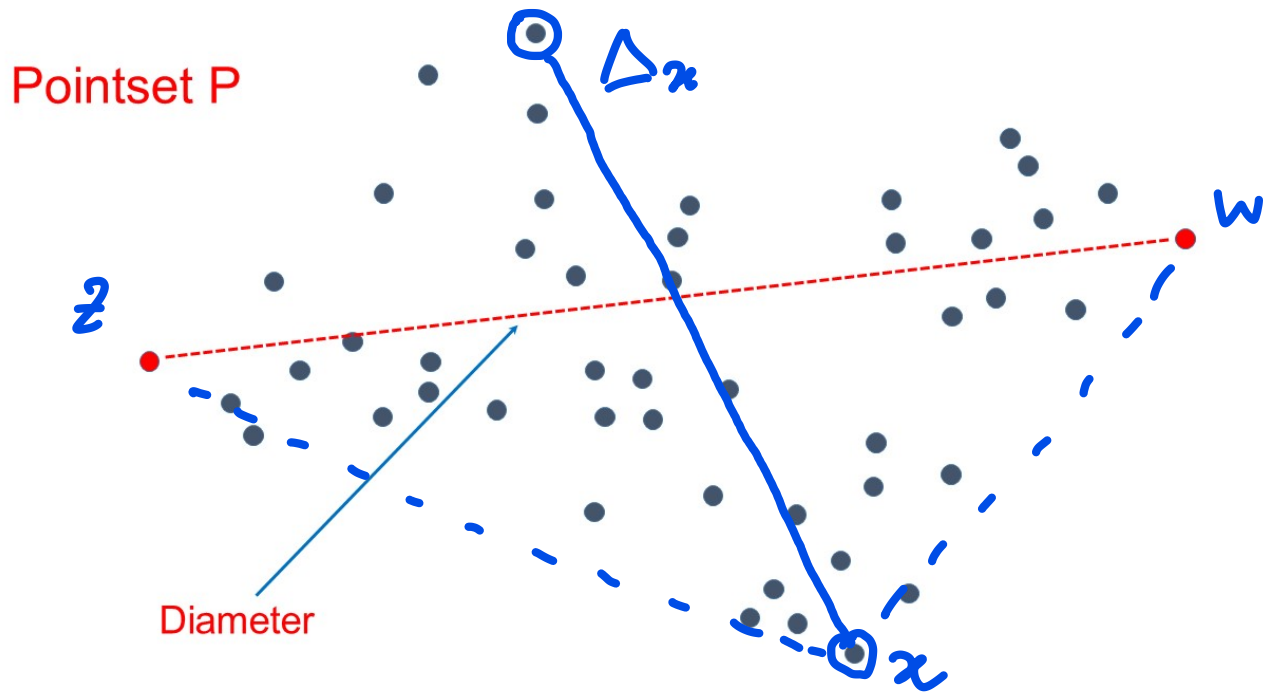
2-approximation to the diameter

For an arbitrary $x \in P$ define

$$\Delta_x = \max\{d(x, y) \ : \ y \in P\}.$$

Lemma

For any $x \in P$ we have $\Delta(P) \in [\Delta_x, 2\Delta_x]$.



Proof

* $\Delta(P) \geq \Delta_x$ since $\Delta(P)$ is the maximum distance over "all" pairs of points, while Δ_x is the maximum distance over all pairs that include x

* $\Delta(P) \leq 2\Delta_x$. Let z, w be such that $\Delta_x = d(z, w)$ Then, by triangle inequality, we have

$$\Delta(P) = d(z, w) \leq d(z, x) + d(x, w) \leq 2\Delta_x$$

□

Exercise

Show that for any arbitrary $x \in P$, the value Δ_x can be computed in MapReduce using 2 rounds, $M_L = O\left(\sqrt{N}\right)$ and $M_A = O(N)$.

Better, coreset-based diameter approximation

Coreset-based approximation:

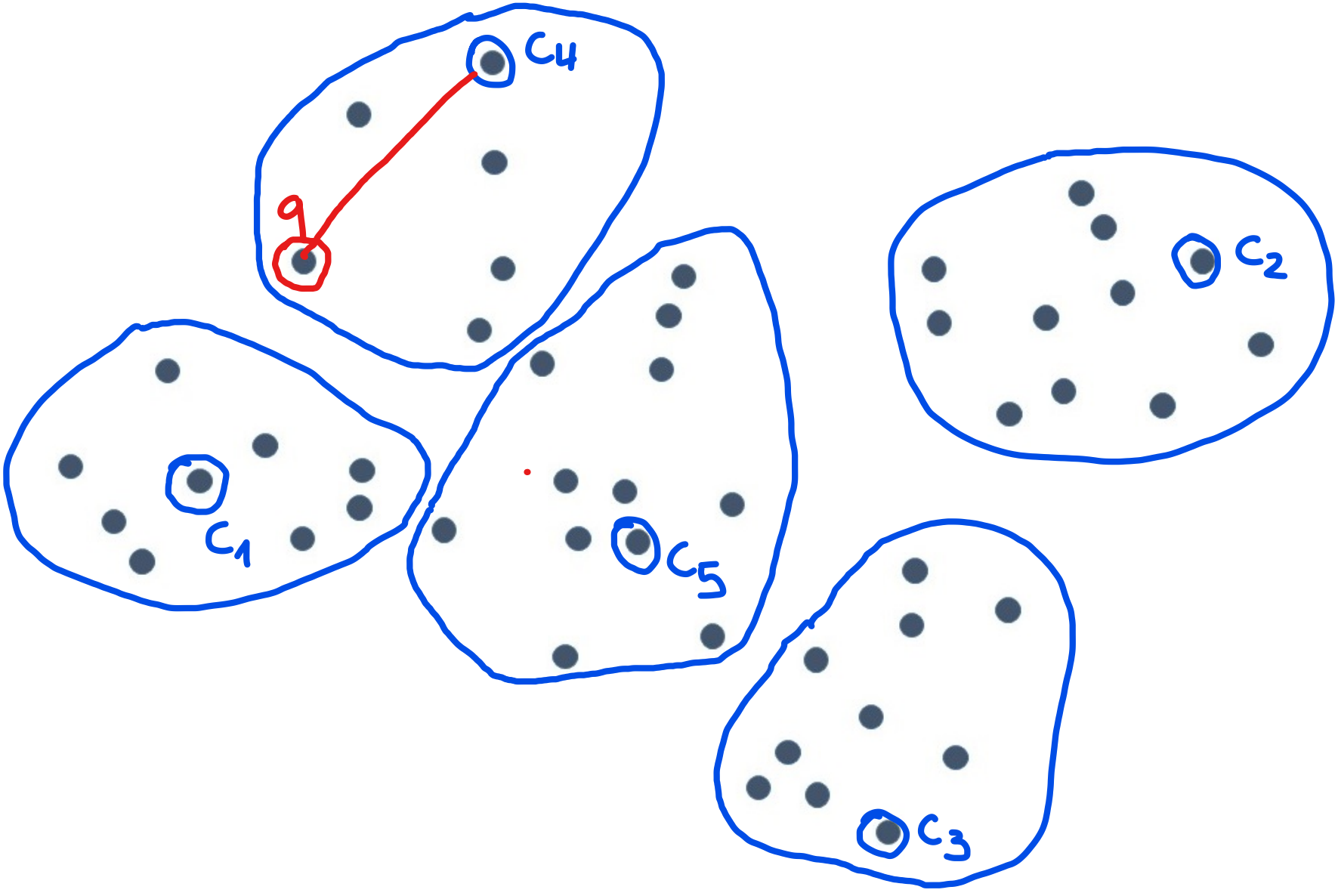
- Fix a suitable $k \geq 2$.
- Extract a coreset $T \subset P$ of size k by running a k-center clustering algorithm on P and taking the k returned centers as set T.
- Return $\Delta(T) = \max_{x,y \in T} d(x,y)$ as an approximation of $\Delta(P)$.

Can the approximation be computed efficiently?

If $k = O(1)$, $\Delta(T)$ can be computed

- Sequentially: in $O(N)$ time (using Farthest-First Traversal)
- In MapReduce: in 2 rounds, with local space $M_L = O\left(\sqrt{N}\right)$ and aggregate space $M_A = O(N)$ (through MR-Farthest-First Traversal)

Is $\Delta(T)$ a good approximation of $\Delta(P)$?


c4
q
c2
c1
c5
c3

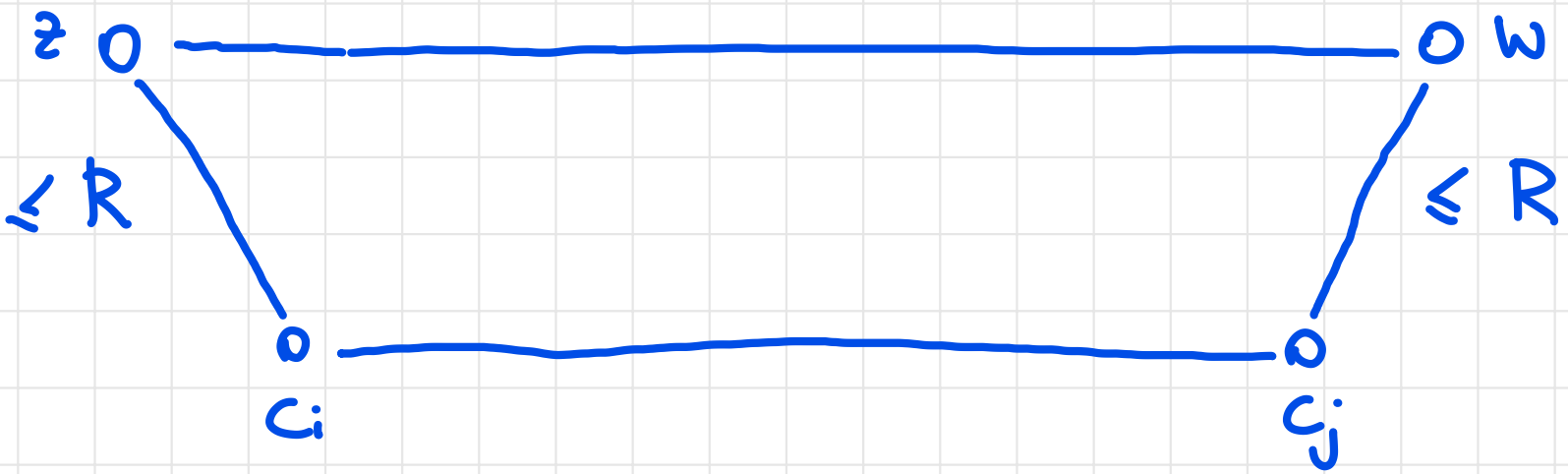
$T = \{c_1, c_2, \ldots, c_k\}$

q = point of P farthest from T

$\Delta(P)$ = true diameter of P : e.g. $\Delta(P) = d(z, w)$

Let c_i = center of T closest to z

c_j = center of T closest to w



$$\Rightarrow \Delta(p) = d(z,w)$$

$$\leq d(z,c_i) + d(c_i,c_j) + d(c_j,w)$$

$$\leq 2R + d(c_i,c_j)$$

$$\leq 2R + \Delta(T)$$

$$\Rightarrow \quad \Delta(T) \leq \Delta(P) \leq \Delta(T) + 2R$$

since $T \subseteq P$

by the above derivation

OBSERVATIONS

* R provides a quantitative estimate of the quality of the approximation, and it can be easily computed while determining the correct T
* It is easy to see that R is non increasing with k. Moreover, for low-dimensional datasets it can be shown that R decreases sharply as k increases

* Suppose that k is chosen so that $R \leq \varepsilon\, d(c_1, c_2)$ where $\varepsilon \in (0,1)$ is an accuracy parameter.

$$\Rightarrow \Delta(P) \leq \Delta(T) + 2R$$

$$\leq \Delta(T) + 2\varepsilon\, \Delta(T)$$

$$= (1 + 2\varepsilon)\, \Delta(T)$$

As $\varepsilon \to 0$ the approximation ratio tends to 1!

Diversity maximization

Objective:

For a given dataset

Diversity maximization

Objective:

Determine the most diverse subset of size k (for a given small k)

Applications

Aggregator websites (e.g., Google News)

- Selection of documents for home page
- Diversified set of representative docs

E-commerce

- Selection of consideration set: products returned by portal upon user query
- Diversity of returned products correlates to user satisfaction

Facility location

- Selection of non-competing franchise store locations
- Selection of strategic facility locations (to avoid simultaneous attacks)

Formal definition of the problem

Diversity maximization problem: max-sum (or remote clique) variant

Given a set P of points from a metric space (M, d) and a positive integer $k < |P|$, return a subset $S \subset P$ of k points, which maximizes the diversity function

$$\mathrm{div}(S) = \sum_{x,y \in S} d(x, y).$$

There are $\binom{k}{2}$ terms in the sum

The problem is also called: MIN-SUM DIVERSIFICATION

Observations:

- NP-hard problem
- A c-approximation algorithm is known with $c = 2 - 2/k$ which, however, requires quadratic time (impractical for large inputs!)
- Other variants: different functions $\mathrm{div}(\cdot)$ to maximize (all NP-hard)

Coreset-based approach to diversity maximization

For a given input P, define the value of the optimal solution as:

$$\mathrm{div}^{\mathrm{opt}}(P,k) = \max_{S \subset P, |S|=k} \mathrm{div}(S).$$

Can we extract a good solution from a small coreset $T \subset P$? Yes, as long as T is a good coreset. The quality of a coreset is quantified as follows.

Definition: $(1+\epsilon)$-coreset

Let $\epsilon \in (0,1)$ be an accuracy parameter. A subset $T \subset P$ is an $(1+\epsilon)$-coreset for the diversity maximization problem on P if

$$\mathrm{div}^{\mathrm{opt}}(T,k) \geq \frac{1}{1+\epsilon}\mathrm{div}^{\mathrm{opt}}(P,k).$$

Obs.: since we are solving a maximization problem, the smaller ϵ the better the optimal solution in T approximates the optimal solution in P.

Note that for any T we have $\mathrm{div}^{\mathrm{opt}}(T,k) \leq \mathrm{div}^{\mathrm{opt}}(P,k)$

The definition implies that if T is a $(1+\varepsilon)$-coreset and if S is a c-approximation to the optimal solution for the diversity maximization on T then S is a $c\cdot(1+\varepsilon)$-approximation to the optimal solution for diversity maximization on P, That is

$$div(S) \geq \frac{1}{c(1+\varepsilon)} div^{OPT}(P,k)$$

Obs. Although we think ε small, it can be > 1

Coreset-based approach to diversity maximization

The following fact, establishes an interesting relation between the k-center and the diversity maximization problems.

Fact

$$\Phi^{\text{opt}}_{\text{kcenter}}(P,k) \leq \frac{\text{div}^{\text{opt}}(P,k)}{\binom{k}{2}}$$

This property, is crucially exploited by the coreset-based approach described (at a high level) in the following slide.

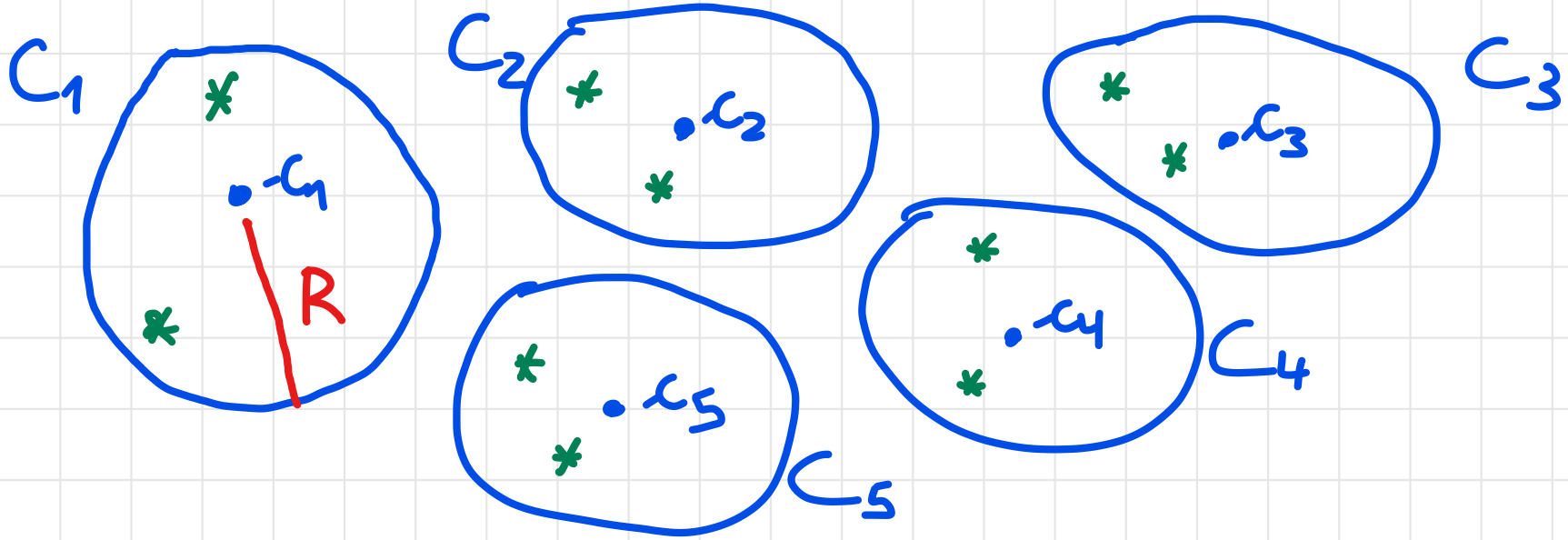
High-level coreset-based algorithm

Given P and k:

- Extract a set $W \subseteq P$ of $h > k$ centers, using Farthest-First Traversal, for a suitable $h > k$.
- Let $P = C_1 \cup C_2 \cup \cdots \cup C_h$ be the clustering induced by W.
- Compute a coreset T of size $\leq h \cdot k$ selecting $\min\{k, |C_i|\}$ points from each C_i (including the center), with $1 \leq i \leq h$.
- Compute the final solution S by running the best sequential algorithm for diversity maximization on T.

- Is this approach effective?
- What is a good choice for h?

Example: $k=3$ $h=5$ ⇒ we will have 5 clusters



R = radius of the clustering

Coreset T = $h \cdot k$ points, obtained selecting from each cluster:
- the center
- $k-1$ additional points

IDEA: let

S^* = optimal solution to diversity maximization on P (i.e., the k points that maximize div())

CRUCIAL OBSERVATION: the fact that we select k points from each cluster, to be included in T, and that a cluster cannot contain more than k points of S^* implies that

$\forall x \in S^* \exists$ a DISTINCT PROXY $\tau(x) \in T$ which belongs to the same cluster as x

$\Rightarrow d(x, \tau(x)) \leq 2R$ (by triangulating with the center of the cluster)

Now, if R is VERY SMALL, the set

$$S = \{\tau(x) : x \in S^*\} \subseteq T$$

Contains k points which are a good "image" of S^* in T. Intuitively, in this case $div(S)$ would not be too far from $div(S^*)$ and since $div(S) \leq div^{OPT}(T, k)$ (because $S \subseteq T$)

this shows that T is a good coreset.

Indeed it can be shown that if

$$R \leq \frac{1}{8} \Phi^{OPT}_{Kmeans}(P,k)$$

then (using the previous fact)

$$div(S) \geq \frac{1}{2} div^{OPT}(P,k)$$

Thus

$$div^{OPT}(T,k) \geq div(S) \geq \frac{1}{2} div^{OPT}(P,k)$$

proving that T is a 2-coreset

that is a $(1+\varepsilon)$- coreset with $\varepsilon = 1$

In general, making R even smaller improves the quality of T, which becomes a $(1+\varepsilon)$-coreset where ε decreases proportionally to R

Answers to the 2 questions (bottom of slide 25)

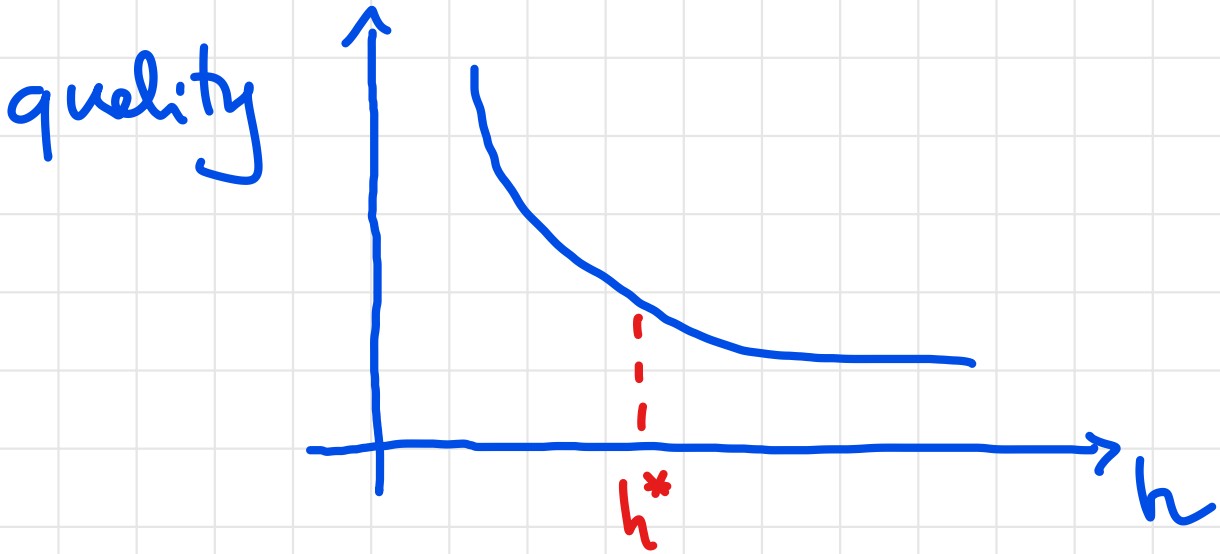
* Is this approach effective? YES

  If h is sufficiently large, hence R is sufficiently small, T becomes a $(1+\varepsilon)$-coreset

with small ε, and running a C-approximation algorithm on T yields a $C \cdot (1+\varepsilon)$-approximate solution (see Slide 23)

* What is a good choice for h? h governs a tradeoff: the larger h the better the approximation but the larger $|T|$ ($\Rightarrow$ higher computation costs)

In practice, since the tradeoff between h and the solution quality is something like



One should pick a value of h (say h^*) close to the elbow point

k-means/median

Definition of the problems

Let us review the two problems

Given a pointset P of N points from a metric space (M, d), determine a set $S \subset P$ of k centers which minimizes

$$\Phi_{\text{kmeans}}(P, S) = \sum_{x \in P} (d(x, S))^2 \qquad \text{(k-means)}$$

$$\Phi_{\text{kmedian}}(P, S) = \sum_{x \in P} (d(x, S)) \qquad \text{(k-median)}$$

Observation: depending on the application and/or the algorithm used, the requirement that S be a subset of P may be lifted, and the centers can be allowed to be aribtrary points of M.

Current popular algorithms for k-means/median

Lloyd's algorithm: (a.k.a. "k-means algorithm")

- APPLICABILITY. Used only for k-means when $M = \mathbb{R}^D$, $d(\cdot, \cdot)$ is the standard Euclidean distance (L_2-distance), and centers can be selected outside P.
- ACCURACY. If initial centers are selected well (e.g., through k-means++) it usually provides good solutions, but, if not, it may be trapped into local optima.
- EFFICIENCY. Efficient implementations are provided by most common software packages, but a limit on the number of iterations is needed in case of very slow convergence.
- SUITABILITY FOR MASSIVE INPUTS. Yes if data can be processed by a distributed platform and only few iterations are executed. However, it is not suitable to process data streams,

Current popular algorithms for k-means/median

k-means++ algorithm: (by Arthur & S. Vassilvitskii, 2007).

$c_1 \leftarrow$ random point chosen from P with uniform probability;
$S \leftarrow \{c_1\}$;
for $2 \leq i \leq k$ **do**
    foreach $x \in P - S$ **do** $\pi(x) \leftarrow (d(x,S))^2 / \sum_{y \in P-S} (d(y,S))^2$;
    $c_i \leftarrow$ random point in $P - S$ according to distribution $\pi(\cdot)$;
    $S \leftarrow S \cup \{c_i\}$
return S

Obs. S is a random variable !

Observation: k-means++ is a randomized algorithm and it is known that in expectation, the returned solution is an α-approximation, with $\alpha = \Theta(\ln k)$.

Precisely: $E[\Phi_{kmeans}(P,S)] \leq \alpha \Phi^{opt}_{kmeans}(P,k)$

↳ expectation over the distribution of S

Current popular algorithms for k-means/median

k-means++ algorithm:

- APPLICABILITY. It can be used for both k-means and k-median, and enforces $S \subset P$.
- ACCURACY. It provides decent solutions, but it is often useful to refine them to get better accuracy.
- EFFICIENCY. Very easy and efficient implementation.
- SUITABILITY FOR MASSIVE INPUTS. Yes if data can be processed by a distributed platform and k is small. However, it is not suitable to process data streams.

Current popular algorithms for k-means/median

Partitioning Around Medoids (PAM) algorithm (a.k.a. k-medoids).

Devised by Kaufman and Rousseeuw in 1987, it is based on a local search strategy which starts from an arbitrary solution S and progressively improves it by performing the best swap between a point in S and a point in $P - S$, until no improving swap exists.

- APPLICABILITY. Mainly used for k-median, and enforces $S \subset P$.
- ACCURACY. It provides good solutions.
- EFFICIENCY. Very slow since each iterations requires checking about $N \cdot k$ possible swaps and the convergence can be very slow.
- SUITABILITY FOR MASSIVE INPUTS. Not at all!

Consequences of the above scenario

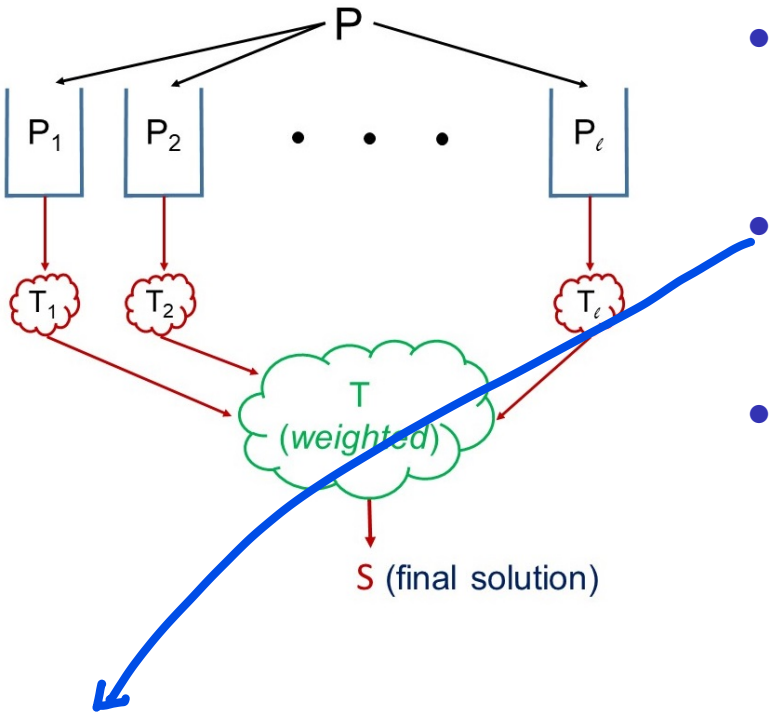
Clustering of massive data remains challenging if

* we need to analyze data from general metrics
* we require very high accuracy (especially in high-dimensional spaces)
* data arrive as possibly unbounded streams

Coreset-based strategies are ESSENTIAL in these scenarios

Coreset-based approach for k-means/median

The composable coreset technique can be employed to devise k-means/median algorithms which are able: (i) to handle massive data (in a distributed setting); and (ii) to provide good accuracy.



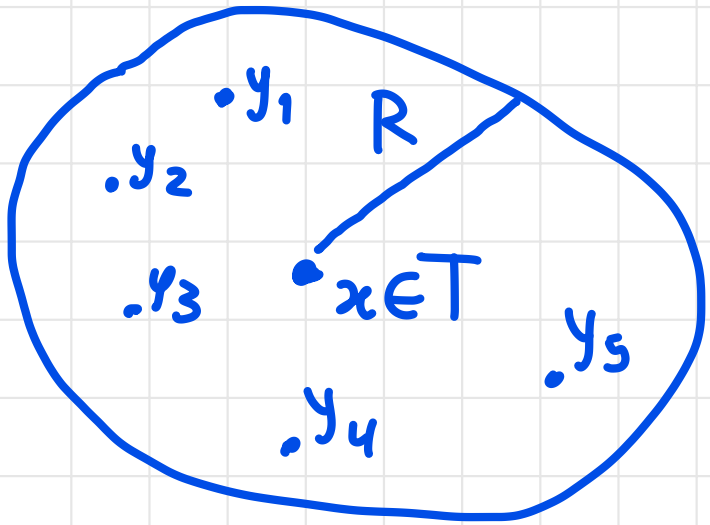
- Each point $x \in T_i$ is given a weight $w(x)$ = the number of points in P_i for which x is the closest representative in T_i.
- The local coresets T_i's are computed using a sequential algorithm for k-means/median.
- The final solution S is also computed using a sequential algorithm for k-means/median, adapted to handle weights.

in other words, w(x) is the size of the cluster centered at x

$T_i \equiv$ set of centers computed on P_i

Why are weights needed?

In the coreset-based approach for k-center/means/median the coreset T for P consists of cluster centers, where each center $x \in T$ represents all points of $P-T$ in its cluster



$y_1, \ldots, y_5$ are the points represented by x

WE ASSUME R very small

Let $S \subseteq T$ be the solution computed from T

For k-center: $d(x, S)$ is a good estimate of the contribution of all y_i's to $\phi_{\text{center}}(P, S)$

For k-means: $w(x) \cdot d(x, S)$ is a good estimate of the contribution of all y_i's to $\phi_{\text{means}}(P, S)$

while, unlike the case of k-center, $d(x, S)$ on its own would not be a good estimate of the contribution of all y_i's and x

⇒ In order to be able to select point a pol solution S for the entire $\mathcal{P}$, w.r.t. k-means clustering, the weights must be taken into account

Same observations apply to k-median

From now on we focus on k-means

What we will say applies as well to k-median
(with very minor adaptations)

Weighted k-means clustering

Weighted variant of the k-means clustering problem

Input:

- Set P of N points from $\mathbb{R}^D$.
- Integer weight $w(x) > 0$ for every $x \in P$.
- Target number k of clusters.

Output: Set S of k centers in $\mathbb{R}^D$ minimizing

$$\Phi^w_{\text{kmeans}}(P, S) = \sum_{x \in P} w(x) \cdot (d(x, S))^2.$$

Observations:

- This formulation allows centers outside P.
- If $w(x) = 1$ for every $x \in P$ we have the standard k-means clustering problem.

How to adapt known algorithms to handle weights

* k-means++

It is sufficient to change the probability of selecting the new center c_i in iterations $i = 2 \ldots$ *

$$\Pi(x) = \frac{d(x,S)^2}{\sum_{y \in P-S} d(y,S)^2} \longrightarrow \frac{w(x) \cdot d(x,S)^2}{\sum_{y \in P-S} w(y)\, d(y,S)^2}$$

* Lloyd's algorithm

- $\Phi_{kmeans}(P,S) \longrightarrow \Phi^{w}_{kmeans}(P,S)$

- Centroid for a cluster $C = \{x_1, x_2, \ldots, x_t\}$

$$\frac{1}{t}\sum_{i=1}^{t} x_i \longrightarrow \frac{1}{\sum_{i=1}^{t} w(x_i)} \cdot \sum_{i=1}^{t} w(x_i)\cdot x_i$$

Obs. For k-medians, PAM algorithm can be adapted to handle weights in a similar fashion

Coreset-based MapReduce algorithm for k-means

MR-kmeans($\mathcal{A}_1, \mathcal{A}_2$): MapReduce algorithm for k-means (described in the next slide) which 2 sequential algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$ for k-means, such that:

- $\mathcal{A}_1$ solves the standard variant without weights
- $\mathcal{A}_2$ solves the more wighted variant
- Both $\mathcal{A}_1$ and $\mathcal{A}_2$ require space proportional to the input size.

Obs. $\mathcal{A}_2$ can be used (with unit weights) in place of $\mathcal{A}_1$

Input Set P of N points in $\mathbb{R}^D$, integer $k > 1$, sequential k-means algorithms $\mathcal{A}_1, \mathcal{A}_2$.

Output Set S of k centers in $\mathbb{R}^D$ which is a good solution to the k-means probelm on P.

MR-kmeans$(\mathcal{A}_1, \mathcal{A}_2)$

Round 1:

- Map Phase: Partition P arbitrarily in ℓ subsets of equal size $P_1, P_2, \ldots, P_\ell$.
- Reduce Phase: for every $i \in [1, \ell]$ separately, run $\mathcal{A}_1$ on P_i to determine a set $T_i \subseteq P_i$ of k centers, and define
  - For each $x \in P_i$, the proxy $\tau(x)$ as x's closest center in T_i.
  - For each $y \in T_i$, the weight $w(y)$ as the number of points of P_i whose proxy is y.

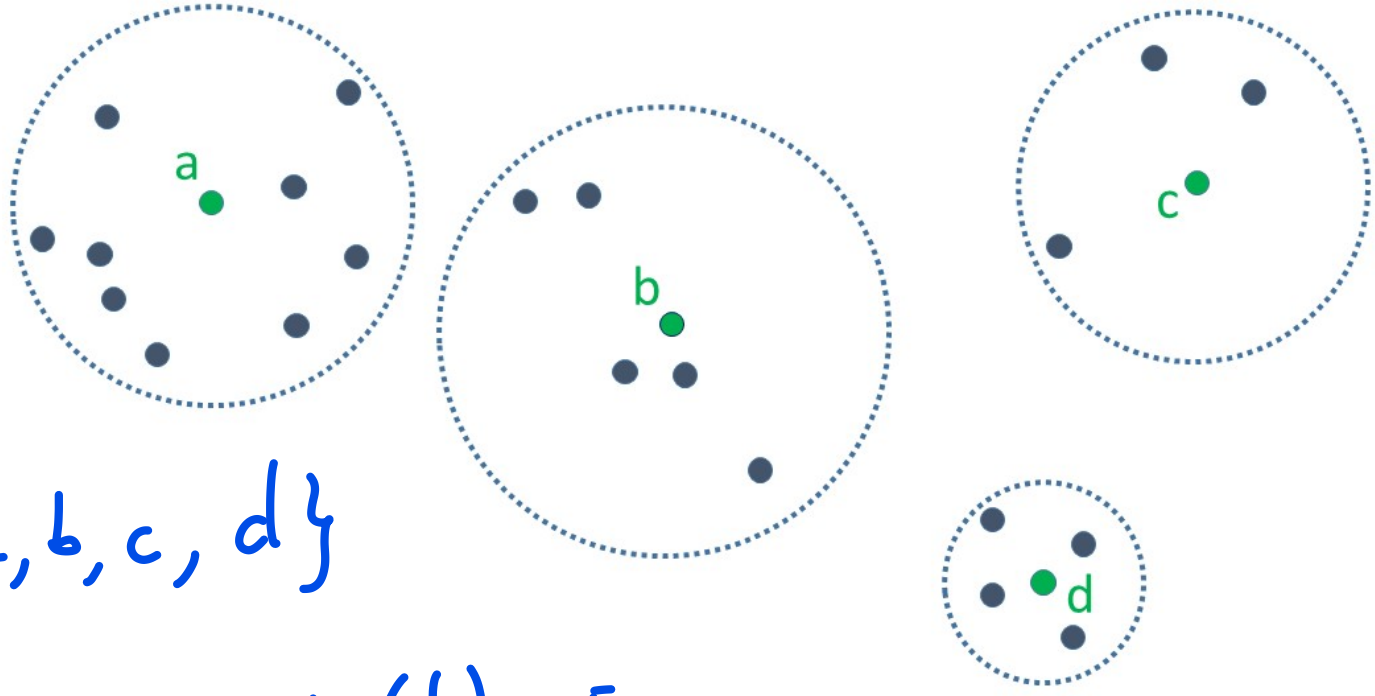
the $\tau(x)$'s must not be stored but the weights $w(y)$'s must be stored

Round 2:

- Map Phase: empty.
- Reduce Phase: run, using a single reducer, $\mathcal{A}_2$ on $T = \cup_{i=1}^{\ell} T_i$ (with the given weights) to determine a set $S = \{c_1, c_2, \ldots, c_k\}$ of k centers, which is then returned as output.

Example for Round 1

Set T_i (green points) computed in a partition P_i



$k = 4$

$T_i = \{a, b, c, d\}$

$w(a) = 10$ $w(d) = 5$

$w(b) = 6$

$w(c) = 4$

Analysis of MR-kmeans$(\mathcal{A}_1, \mathcal{A}_2)$: performance

$k = o(N)$

Assume $k \leq \sqrt{N}$. By setting $\ell = \sqrt{N/k}$, it is easy to see that MR-kmeans$(\mathcal{A}_1, \mathcal{A}_2)$ requires

- Local space $M_L = O\left(\max\{N/\ell, \ell \cdot k\}\right) = O\left(\sqrt{N \cdot k}\right) = o(N)$
- Aggregate space $M_A = O\left(N\right)$

Same analysis as MR-FFT

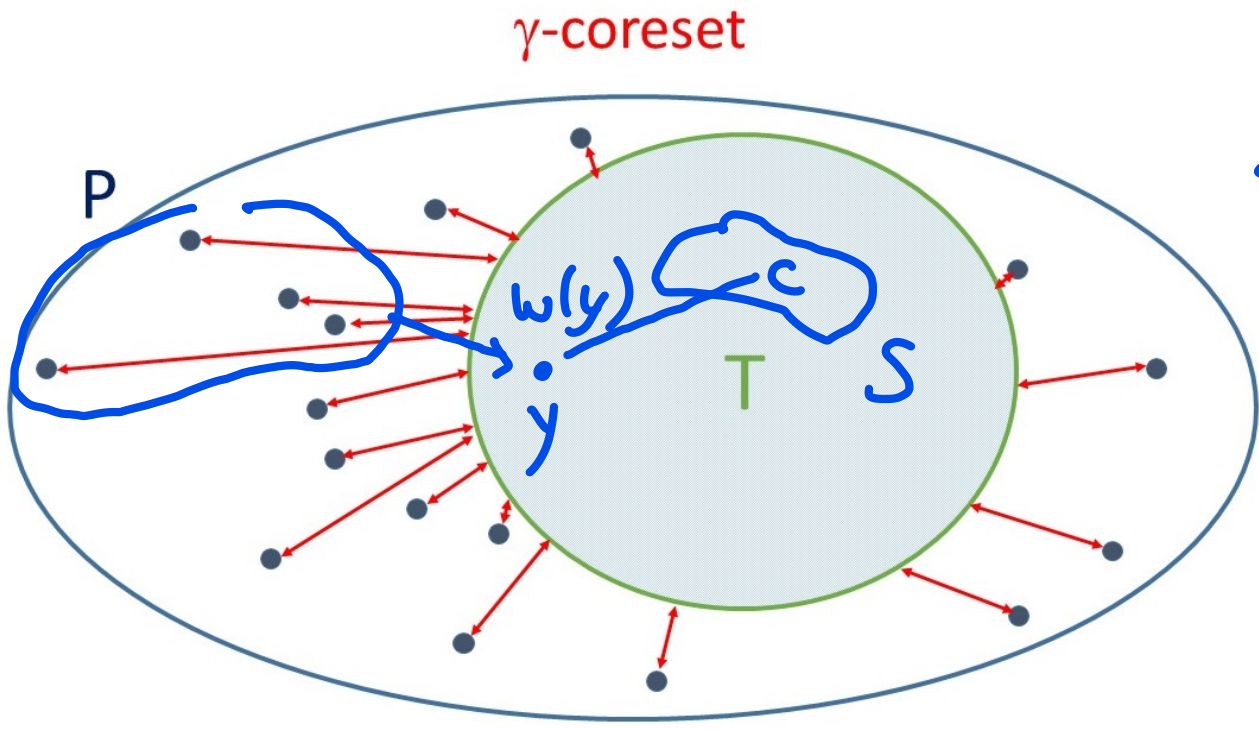
Analysis of MR-kmeans$(\mathcal{A}_1, \mathcal{A}_2)$: accuracy

In order to analyze the quality of the solutions computed by MR-kmeans$(\mathcal{A}_1, \mathcal{A}_2)$ we need to introduce the following notion:

Definition

Given a pointset P, a coreset $T \subseteq P$ and a proxy function $\tau : P \to T$, we say that T is a γ-coreset for P, k and the k-means objective if

$$\sum_{p \in P} (d(p, \tau(p)))^2 \leq \gamma \cdot \Phi^{\text{opt}}_{\text{kmeans}}(P, k).$$



S = solution computed on T using weights

Sum of squared distances from P-T to T (red lines) $\leq \gamma \, \Phi^{opt}_{kmeans}(P,k)$

Essentially, if T is a γ-coreset with small γ, we know that the error that we make approximating $\Phi_{kmeans}(P,S)$ with $\Phi^{w}_{kmeans}(T,S)$ is small

Theorem

Suppose that

- $\mathcal{A}_1$ *is a* γ*-approximation algorithm for the unweighted k-means problem;*
- $\mathcal{A}_2$ *is an* α*-approximation algorithm for the weighted k-means problem.*

Then:

- *The coreset* T *computed in Round 1 is a* γ*-coreset.*
- *The solution* S *computed in Round 2 is such that*

$$\Phi_{\text{kmeans}}(P, S) = O\left((1+\gamma)\cdot\alpha\right)\cdot\Phi^{\text{opt}}_{\text{kmeans}}(P, k).$$

We skip the proof. If interested, you can find it in [MPP19].

The formula for the final approximation ratio contains 2 contributions to the loss of accuracy

- The loss of accuracy coming from the fact that $T \subseteq P$ ($(1+\gamma)$ term)
- The loss of accuracy coming from using an approximation algorithm to compute the final solution S from T (α term)

Observations on MR-kmeans$(\mathcal{A}_1, \mathcal{A}_2)$

- For k-means (as well as for k-center and k-median) good coresets are obtained by combining solutions to the same problem on smaller partitions. However, this is not always the case for other problems (e.g., diameter, diversity maximization).
- In practice, the algorithm is fast and accurate.
- Typically, one could use k-means++ as $\mathcal{A}_1$ in Round 1, and a weighted version of k-means++ plus LLoyd's as $\mathcal{A}_2$ in Round 2.
- If $h > k$ centers are selected from each P_i in Round 1 (e.g., through k-means++), the quality of T, hence the quality of the final clustering, improves. In fact, for low dimensional spaces, a γ-coreset T with $\gamma \ll \alpha$ can be built in this fashion using a value h not too large.

Exercises

Exercise

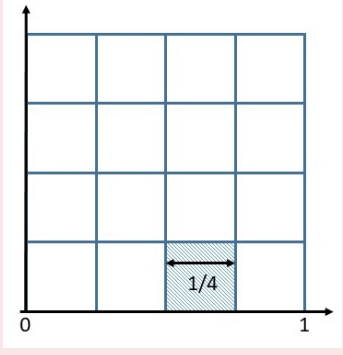
Recall that k-means++ is a randomized algorithm and the quality of the solution is a random variable. Letting S be the set of centers returned by k-means++ for P it is known that there is a value $\alpha = \Theta(\ln k)$ such that

$$\Pr\left(\Phi_{\text{kmeans}}(P, S) \leq \alpha \cdot \Phi^{\text{opt}}_{\text{kmeans}}(P, k)\right) \quad \geq \quad 1/2.$$

Show that the above probability can be made $\geq 1 - 1/N$, if we run several *independent instances* of k-means++, and return the set of centers which yields the minimum value of the objective function, among the ones computed in the various runs.

Exercise

Let Q be the unit square in $\mathbb{R}^2$, with bottom-left corner $(0, 0)$, and consider Q subdivided into c^2 square *cells* of side $1/c$. Below is an example for $c = 4$.



Consider a set of points P from Q and a subset $T \subset P$ of c^2 points, which contains one point from each cell (assume one always exists). Let $d_{\max}(P)$ be the maximum pairwise distance between any 2 points of P and $d_{\max}(T)$ the maximum pairwise distance between any 2 points of T. Quantify how well $d_{\max}(T)$ approximates $d_{\max}(P)$.

Exercise

Let P be a set of N weighted points partitioned into k clusters $C_1, C_2, \ldots, C_k$ with respective centers $c_1, c_2, \ldots, c_k$. Each point $x \in P$ is represented by the pair $(x, (i_x, w_x))$, where i_x is the index of the cluster of x, and w_x is the weight of x. For every cluster C_i its *weighted closest neighbor* is the cluster C_j, with $j \neq i$, which *minimizes* the following sum

$$\sum_{x \in C_i} w_x \cdot d^2(x, c_j) \qquad (c_j \text{ center of } C_j).$$

1. Design an efficient *2-round MapReduce algorithm* that for each $1 \leq i \leq k$ computes the index j of C_i's closest neighbor. *You can assume k constant and that $c_1, \ldots c_k$ to be global variables.*
2. Analyze the local and aggregate space required by your algorithm. For full score, your algorithm must require $o(N)$ local space and $O(N)$ aggregate space.

Summary of Parts 1 and 2

- (Composable) coreset technique.
- Metric space and prominent distance functions.
- Combinatorial optimization problem and approximation algorithm.
- k-center clustering:
  - Definition of the problem.
  - Farthest-First Traversal algorithm.
  - MR-Farthest-First Traversal.
  - k-center as a coreset primitive: diameter, diversity maximization.
- k-means/median clustering:
  - Definition of the problem.
  - Critical review of known algorithms.
  - Weighted variant of the problem.
  - MR-kmeans

References

LRU14 J. Leskovec, A. Rajaraman and J. Ullman. Mining Massive Datasets. Cambridge University Press, 2014. Sections 3.1.1 and 3.5, and Chapter 7

AV07 D. Arthur, S. Vassilvitskii. k-means++: the advantages of careful seeding. Proc. of ACM-SIAM SODA 2007: 1027-1035.

CPPU17 M. Ceccarello, A. Pietracaprina, G. Pucci, E. Upfal. MapReduce and Streaming Algorithms for Diversity Maximization in Metric Spaces of Bounded Doubling Dimension. Proc. VLDB Endow. 10(5): 469-480 (2017)

MPP19 A. Mazzetto, A. Pietracaprina, G. Pucci. Accurate MapReduce Algorithms for k-Median and k-Means in General Metric Spaces. ISAAC 2019: 34:1-34:16